TRUST-LED STUDENT AND PROFESSIONAL HOUSING

Find a place. Know what to trust.

SmartStay is a rental marketplace prototype for students and working professionals looking for PGs, hostels, rooms, and homes in cities including Ghaziabad, Noida, Greater Noida, Delhi, and Bengaluru.

The problem

Online rental listings can be stale, misleading, or hard to assess. A tenant may spend time contacting a property that is no longer available, discover that photos or prices differ from reality, or have little way to report the problem.

The idea

SmartStay combines property discovery with listing review, availability updates, and reporting. Owners maintain their listings, admins review listing decisions and activity, and tenants search and flag concerns.

01

Tenant

Search by place, budget, property details, and availability; review listing status and report concerns.

02

Owner

Create and manage property listings, update units, and confirm availability.

03

Administrator

Review listings, approve or reject them, inspect reports, and see audit activity.

Goal

More informed choices

Make trust signals and listing status more visible during the rental search.

How the platform works

The main journey links property submission, review, discovery, availability maintenance, and user feedback.

01 **Owner creates a listing.** Property information, rent, location, type, images, and related listing details are submitted through the owner experience.

02 **The listing enters review.** New backend-created properties start as `PENDING`. Admins can approve or reject; a rejection requires a reason, and the decision is recorded with an audit entry and owner notification.

03 **Tenant searches public listings.** The API returns verified public listings and supports search and filters such as location, rent, gender, sharing, amenities, and availability.

04 **Owner manages units and availability.** Unit states include available, occupied, reserved, and confirmation required. Availability confirmations are time-stamped by the server.

05 **Freshness gives availability context.** SmartStay calculates a freshness status from the latest confirmation rather than storing a fixed score.

06 **Users report problems.** Reports and review moderation support investigation of concerns such as already-rented listings, misleading photos, or price discrepancies.

Availability freshness bands

Fresh: through 2 hours. **Recently confirmed:** through 6 hours. **Confirmation recommended:** through 12 hours. **Confirmation required:** through 24 hours. After that, the status is **stale**.

These are default backend thresholds and can be configured. Recent availability-related reports can reduce confidence; occupied, reserved, and confirmation-required units are treated as unavailable.

Supporting experiences

StayBot

Answers common SmartStay questions and offers location/budget suggestions using a built-in local knowledge flow; an optional Gemini call is also present.

Admin operations

Admin views include property management, verification review, audit logs, reports, and review moderation.

A freshness status helps communicate recency; it does not guarantee that a room will remain available.

What it is built with

Frontend: static HTML, CSS, and JavaScript pages. **Backend:** Node.js with Express. **Database:** MySQL 8+ using mysql2 and ordered SQL migrations.

Web experience

Separate homepage, property search, tenant, owner, and admin pages. Shared client modules centralize API requests and demo behavior.

REST API

Express routes cover health, authentication, public properties, owner property/unit operations, admin review, audit logs, and verification status.

Persistence

MySQL schema covers accounts, sessions, properties, verification records, units, availability events, reports, notifications, and audit logs.

Security controls

Role guards protect owner/admin routes. Password hashing, HttpOnly session cookies, hashed session tokens, CORS controls, Helmet, and rate limiting are part of the backend dependencies/design.

API and integrations

- **SmartStay REST API:** the primary application API, consumed by the browser client.
- **Gemini API:** optional StayBot integration using the Gemini 1.5 Flash endpoint; local responses remain available without it.
- **DigiLocker:** provider structure and sandbox simulation exist, but production OAuth/document integration is not configured.

Two operating modes

BACKEND MODE

Configured sessions and MySQL support persistent API-backed flows. Backend setup requires Node.js 20+ and MySQL 8+.

LOCAL DEMO MODE

Some page flows use browser localStorage and mock data without a backend. Demo data is browser-local, not shared across devices, and not production-secure.

Authentication in brief

Registration, login, logout, and session lookup use the auth API. Successful login sets an HttpOnly `smartstay.sid` cookie; the database stores a hash of the session token. Public signup cannot create administrator accounts.

Research gap and project contribution

Research gap to investigate

Rental platforms commonly support listing search, but tenants may still have difficulty assessing whether a listing is current, accurately represented, and reviewed. This can be especially consequential for students searching unfamiliar areas.

SmartStay's proposed response

- Connect listing review and tenant discovery in one role-based workflow.
- Expose availability recency through a freshness calculation and timestamped owner confirmations.
- Support unit-level availability rather than relying only on property-level status.
- Provide reporting and admin audit flows for concerns that need investigation.
- Tailor search and guidance to student housing locations and budgets.

Careful differentiation: these features form SmartStay's project contribution and intended focus. They should not be presented as unique across every existing rental platform without a competitor study.

Current boundaries and next research

Prototype boundary: some dashboard areas still use demo data; localStorage state is not shared across browsers.

Verification boundary: the DigiLocker flow simulates consent and outcomes. It does not contact DigiLocker or independently prove legal ownership.

Evaluation needed: interview tenants and owners; measure outdated-listing incidence, time-to-confirm availability, and user trust before and after freshness signals.

Comparison needed: compare real rental platforms using a transparent feature matrix and documented sources.

Production hardening: complete approved identity-provider integration, privacy/security review, deployment controls, and end-to-end testing.

API key handling: the optional Gemini key is currently entered/stored client-side for the demo. A production deployment should proxy requests through a secured backend and keep credentials server-side.

Short presentation script

> "SmartStay is a student-focused rental platform prototype. It addresses a trust and availability-information problem by connecting owner listing management, admin review, tenant search, freshness signals, and issue reporting. The backend uses Express and MySQL; the current demo also has local browser flows. Our next step is to evaluate whether these signals actually help tenants make more confident decisions."

A rigorous research claim needs evidence: literature review, competitor analysis, and feedback or usability testing with target users.